



LARAVEL 5

BLADE TEMPLATES

Contents

- Introduction
- Defining Sections with @section/@show and @yield
- Display Data
- Control Structures
- Components
- Slots
- Including Subviews

Introduction

- Laravel offers a custom templating engine called Blade, which is inspired by .NET's Razor engine. It boasts a concise syntax, a shallow learning curve, a powerful and intuitive inheritance model, and easy extensibility
- Blade does not restrict you from using plain PHP code in your views
- The advantage of Blade views is that they come with automatic caching. All Blade view pages are compiled into plain PHP code and cached until you modify them. This means zero overhead for the application
- Blade view files use the .blade.php file extension and are typically stored in the resources/views directory
- The blade templating engine provides its own structure such as conditional statements and loops



Defining Sections with @section/@show and @yield (1)

- Blade provides a structure for template inheritance that allows views to extend, modify, and include other views.

```
<!-- resources/views/layouts/master.blade.php -->
<html>
  <head>
    <title>My Site | @yield('title', 'Home Page')</title>
  </head>
  <body>
    <div class="container">
      @yield('content')
    </div>
    @section('footerScripts')
      <script src="app.js"></script>
    @show
  </body>
</html>
```



Defining Sections with @section/@show and @yield (2)

```
<!-- resources/views/dashboard.blade.php -->
@extends('layouts.master')

@section('title', 'Dashboard')

@section('content')
    Welcome to your application dashboard!
@endsection

@section('footerScripts')
    @parent
    <script src="dashboard.js"></script>
@endsection
```



Defining Sections with @section/ @show and @yield (3)

- The @yield directive also accepts a default value as its second parameter. This value will be rendered if the section being yielded is undefined

```
@yield('content', View::make('view.name'))
```

```
{{-- This comment will not be present in the rendered HTML --}}
```



Displaying Data

```
Route::get('greeting', function () {  
    return view('welcome', ['name' => 'Samantha']);  
});
```

```
Hello, {{ $name }}.
```

- Blade {{ }} statements are automatically sent through PHP's `htmlspecialchars` function to prevent XSS attacks.
- If you do not want your data to be escaped, you may use the following syntax:

```
Hello, {!! $name !!}.
```



Control Structures (1)

■ If Statements

```
@if (count($records) === 1)
    I have one record!
@elseif (count($records) > 1)
    I have multiple records!
@else
    I don't have any records!
@endif
```

```
@unless (Auth::check())
    You are not signed in.
@endunless
```

```
@isset($records)
    // $records is defined and is not null...
@endisset
@empty($records)
    // $records is "empty"...
@endempty
```



Control Structures (2)

- Section Directives

- You may check if a section has content using the `@hasSection` directive:

```
@hasSection('navigation')
  <div class="pull-right">
    @yield('navigation')
  </div>
  <div class="clearfix"></div>
@endif
```

- You may use the `sectionMissing` directive to determine if a section does not have content:

```
@sectionMissing('navigation')
  <div class="pull-right">
    @include('default-navigation')
  </div>
@endif
```



Control Structures (3)

■ Switch Statements

```
@switch($i)
  @case(1)
    First case...
    @break

  @case(2)
    Second case...
    @break

  @default
    Default case...
@endswitch
```

Loops (1)

```
@for ($i = 0; $i < 10; $i++)
```

```
    The current value is {{ $i }}
```

```
@endfor
```

```
@foreach ($users as $user)
```

```
    <p>This is user {{ $user->id }}</p>
```

```
@endforeach
```

```
@forelse ($users as $user)
```

```
    <li>{{ $user->name }}</li>
```

```
@empty
```

```
    <p>No users</p>
```

```
@endforelse
```

```
@while (true)
```

```
    <p>I'm looping forever.</p>
```

```
@endwhile
```



Loops (2)

```
@foreach ($users as $user)
    @if ($user->type == 1)
        @continue
    @endif

    <li>{{ $user->name }}</li>

    @if ($user->number == 5)
        @break
    @endif
@endforeach
```

- When using loops you may also end the loop or skip the current iteration:

```
@continue($user->type == 1)
```

```
@break($user->number == 5)
```



The Loop Variable – Loops (3)

Property	Description
<code>\$loop->index</code>	The index of the current loop iteration (starts at 0).
<code>\$loop->iteration</code>	The current loop iteration (starts at 1).
<code>\$loop->remaining</code>	The iterations remaining in the loop.
<code>\$loop->count</code>	The total number of items in the array being iterated.
<code>\$loop->first</code>	Whether this is the first iteration through the loop.
<code>\$loop->last</code>	Whether this is the last iteration through the loop.
<code>\$loop->even</code>	Whether this is an even iteration through the loop.
<code>\$loop->odd</code>	Whether this is an odd iteration through the loop.
<code>\$loop->depth</code>	The nesting level of the current loop.
<code>\$loop->parent</code>	When in a nested loop, the parent's loop variable.



The Loop Variable – Loops (4)

```
@foreach ($users as $user)
    @if ($loop->first)
        This is the first iteration.
    @endif
    @if ($loop->last)
        This is the last iteration.
    @endif
    <p>This is user {{ $user->id }}</p>
@endforeach
```

```
@foreach ($users as $user)
    @foreach ($user->posts as $post)
        @if ($loop->parent->first)
            This is first iteration of the
            parent loop.
        @endif
    @endforeach
@endforeach
```



PHP

- In some situations, it's useful to embed PHP code into your views. You can use the Blade `@php` directive to execute a block of plain PHP within your template:

```
@php  
    //  
@endphp
```



Forms

```
<form method="POST" action="/profile">  
    @csrf  
  
    ...  
</form>
```

- Since HTML forms can't make PUT, PATCH, or DELETE requests, you will need to add a hidden `_method` field to spoof these HTTP verbs. The `@method` Blade directive can create this field for you:

```
<form action="/foo/bar" method="POST">  
    @method('PUT')  
  
    ...  
</form>
```



Components

- To create a class based component, you may use the **make:component** Artisan command. The **make:component** command will place the component in the **App\View\Components** directory. The view will be placed in the **resources/views/components** directory.

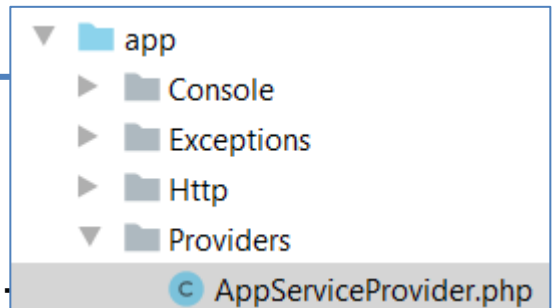
```
php artisan make:component Alert
```

- When writing components for your own application, components are automatically discovered within the **app/View/Components** directory and **resources/views/components** directory.

- Registering Package Components

```
use Illuminate\Support\Facades\Blade;  
public function boot()  
{  
    Blade::component('package-alert', Alert::class);  
}
```

<x-alert/>



<x-package-alert/>



Displaying Components

- To display a component, you may use a Blade component tag within one of your Blade templates. Blade component tags start with the string `x-` followed by the kebab case name of the component class.
- If the component class is nested deeper within the `App\View\Components` directory, you may use the `.` character to indicate directory nesting

```
<x-alert/>
```

```
<x-user-profile/>
```

```
<x-inputs.button/>
```

```
App\View\Components\Inputs\Button.php
```



Passing Data To Components

- Hard-coded, primitive values may be passed to the component using simple HTML attributes. PHP expressions and variables should be passed to the component via attributes that are prefixed with “:”

```
<x-alert type="error" :message="$message"/>
```

```
public function __construct($alertType) {  
    $this->alertType = $alertType;  
}
```

```
<x-alert alert-type="danger" />
```



Component Methods

```
public function isSelected($option) {  
    return $option === $this->selected;  
}
```

```
<option {{ $isSelected($value) ? 'selected="selected"' : " }}"  
    value="{{ $value }}"> {{ $label }}  
</option>
```



Using Attributes & Slots Inside The Class

- Blade components also allow you to access the component name, attributes, and slot inside the class's render method

```
public function render()  
{  
    return function (array $data) {  
        // $data['componentName'];  
        // $data['attributes'];  
        // $data['slot'];  
  
        return '<div>Component content</div>';  
    };  
}
```

```
public function __construct(AlertCreator $creator, $type, $message) {  
    $this->creator = $creator;  
    $this->type = $type;  
    $this->message = $message;  
}
```



Slots

```
<!-- /resources/views/components/alert.blade.php -->  
<div class="alert alert-danger">  
    {{ $slot }}  
</div>
```

```
<x-alert>  
    <strong>Whoops!</strong>  
    Something went wrong!  
</x-alert>
```

```
<!-- /resources/views/components/alert.blade.php -->  
<span class="alert-title">{{ $title }}</span>  
<div class="alert alert-danger"> {{ $slot }} </div>
```

```
<x-alert>  
    <x-slot name="title"> Server Error </x-slot>  
    <strong>Whoops!</strong> Something went wrong!  
</x-alert>
```



Scoped Slots

- You may achieve similar behavior in Laravel by defining public methods or properties on your component and accessing the component within your slot via the `$component` variable

```
<x-alert>
  <x-slot name="title">
    {{ $component->formatAlert('Server Error') }}
  </x-slot>
  <strong>Whoops!</strong> Something went wrong!
</x-alert>
```



Including Subviews

```
@include('view.name', ['some' => 'data'])
```

- If you attempt to @include a view which does not exist, Laravel will throw an error. If you would like to include a view that may or may not be present, you should use the @includeIf directive:

```
@includeIf('view.name', ['some' => 'data'])
```

- If you would like to @include a view if a given **boolean expression evaluates to true**, you may use the @includeWhen directive:

```
@includeWhen($boolean, 'view.name', ['some' => 'data'])
```

- If you would like to @include a view if a given **boolean expression evaluates to false**, you may use the @includeUnless directive:

```
@includeUnless($boolean, 'view.name', ['some' => 'data'])
```

- To include the first view that exists from a given array of views, you may use the includeFirst directive:

```
@includeFirst(['custom.admin', 'admin'], ['some' => 'data'])
```



HOA SEN
UNIVERSITY

www.hoasen.edu.vn

Q&A